

Chapter 4: Neural Network

Basics and Applications of Deep Learning

Fall 2026

Department of Mathematical Data Science

Hanyang University ERICA

Lecturer: Sehyun Kwon



Neural networks: the original linear classifier

(Before) Linear score function: $f = Wx$

$$x \in \mathbb{R}^D, \quad W \in \mathbb{R}^{C \times D}$$

Neural networks: 2 layers

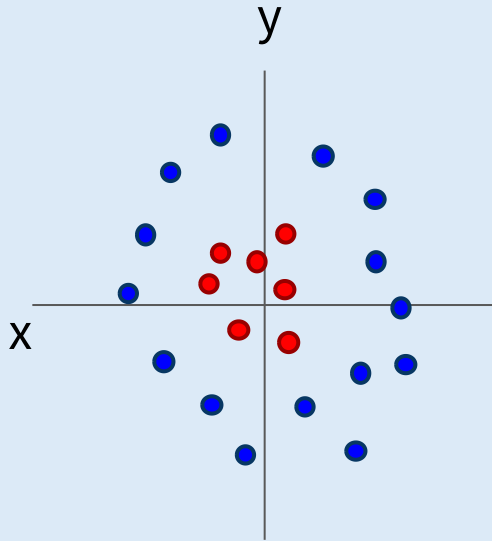
(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

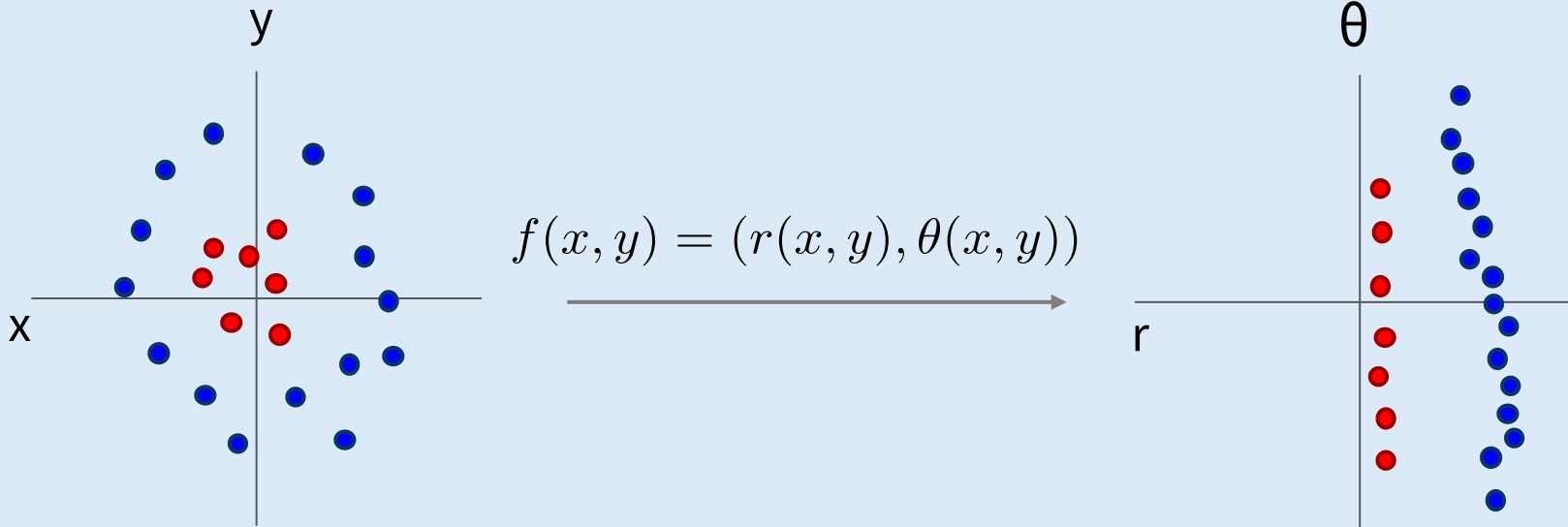
(In practice we will usually add a learnable bias at each layer as well)

Why do we want non-linearity?



Cannot separate red and blue points with linear classifier.

Why do we want non-linearity?



Cannot separate red and blue points with linear classifier.

After applying feature transform, points can be separated by linear classifier.

Neural networks: also called fully connected network

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

“Neural Network” is a very broad term; these are more accurately called “fully-connected networks” or sometimes “multi-layer perceptrons” (MLP)

(In practice we will usually add a learnable bias at each layer as well)

Neural networks: 3 layers

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

or 3-layer Neural Network:

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

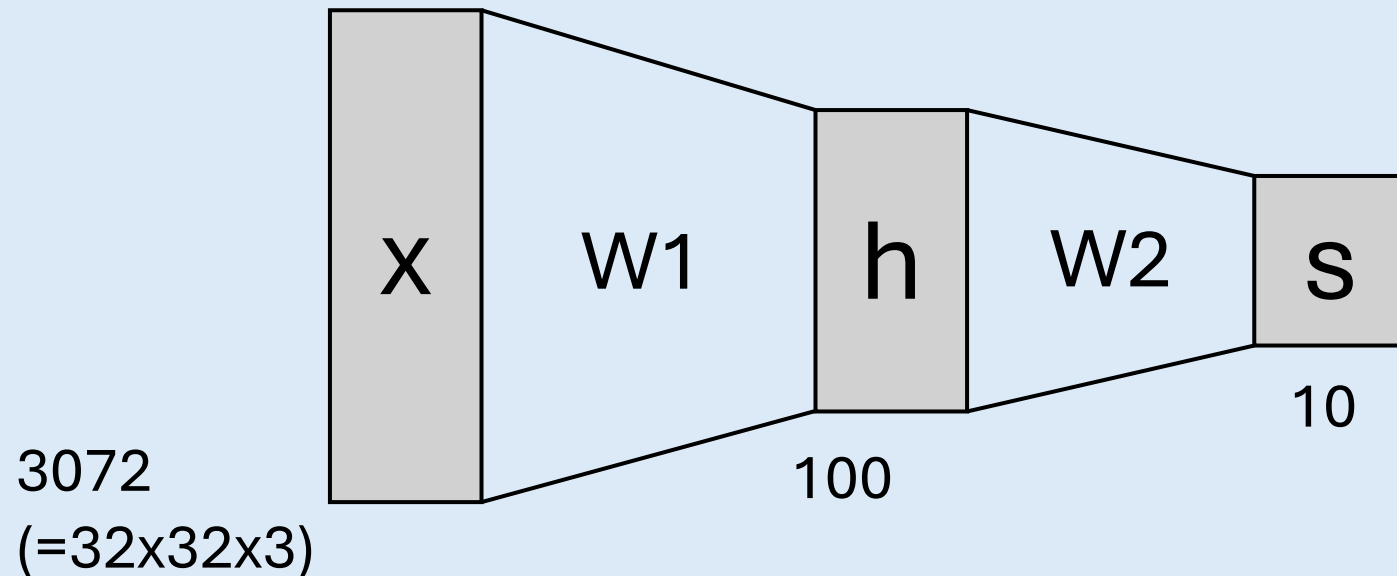
$$x \in \mathbb{R}^D, \quad W_1 \in \mathbb{R}^{H_1 \times D}, \quad W_2 \in \mathbb{R}^{H_2 \times H_1}, \quad W_3 \in \mathbb{R}^{C \times H_2}$$

(In practice we will usually add a learnable bias at each layer as well)

Neural networks: hierarchical computation

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Neural networks: hierarchical computation

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

The function $\max(0, z)$ is called the activation function.

Q: What if we try to build a neural network without one?

Neural networks: hierarchical computation

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

The function $\max(0, z)$ is called the activation function.

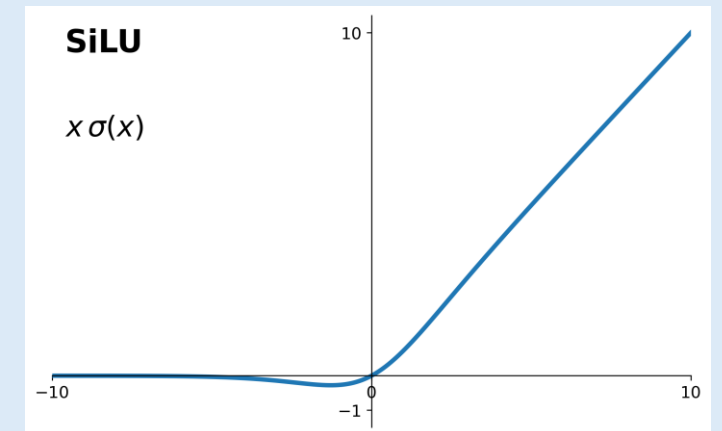
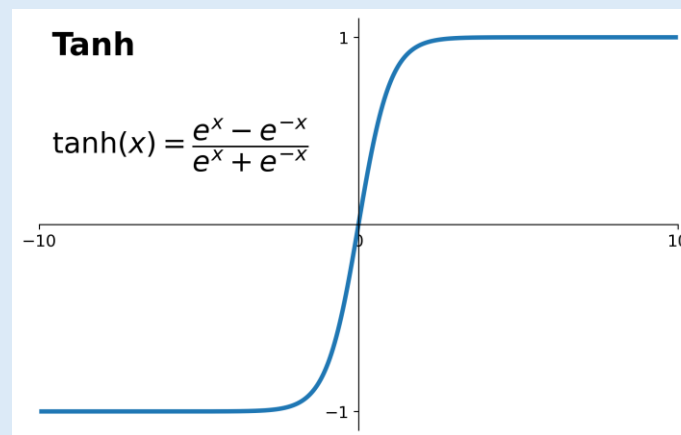
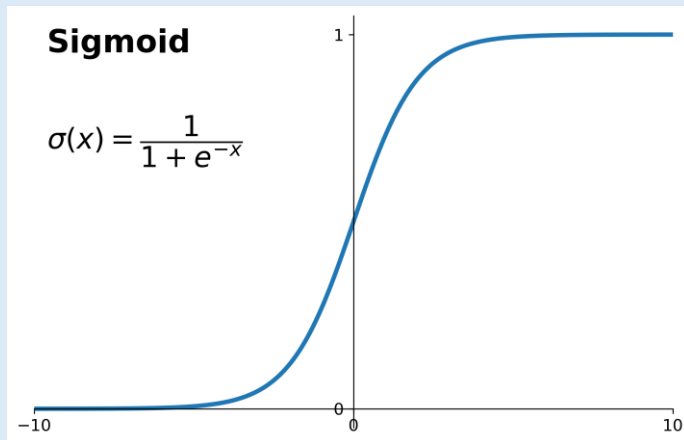
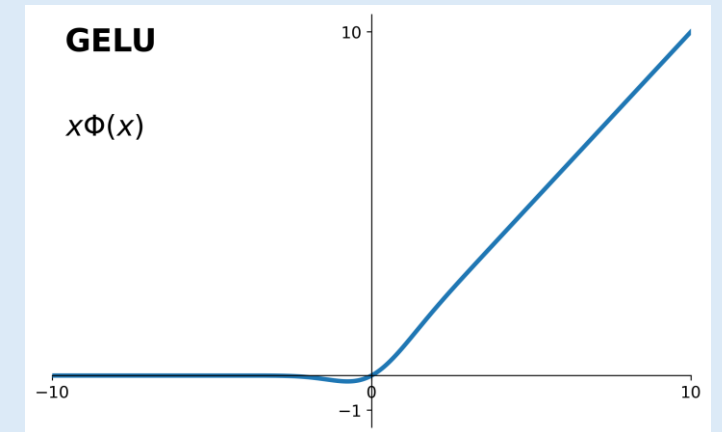
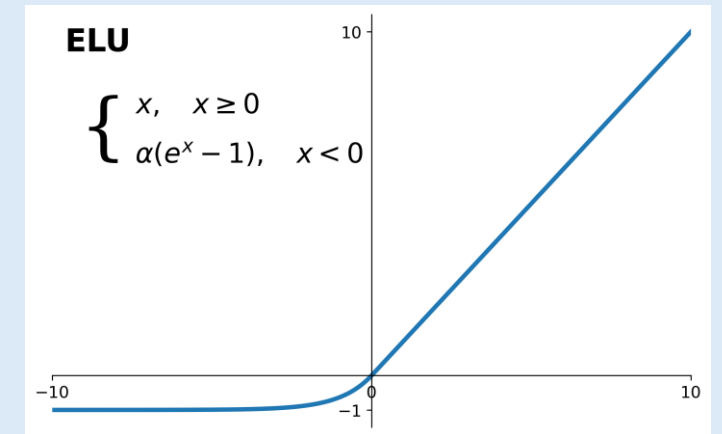
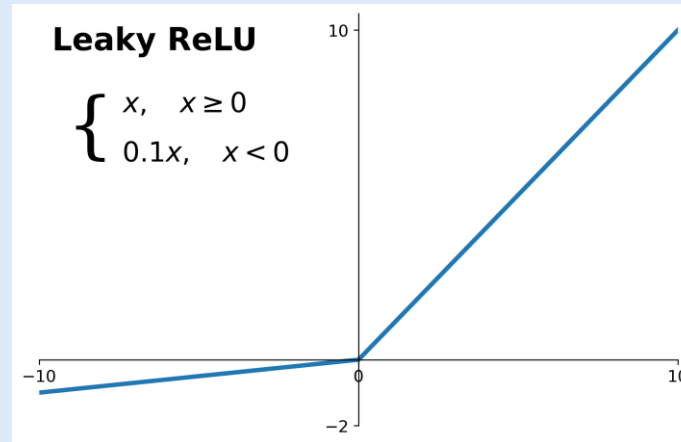
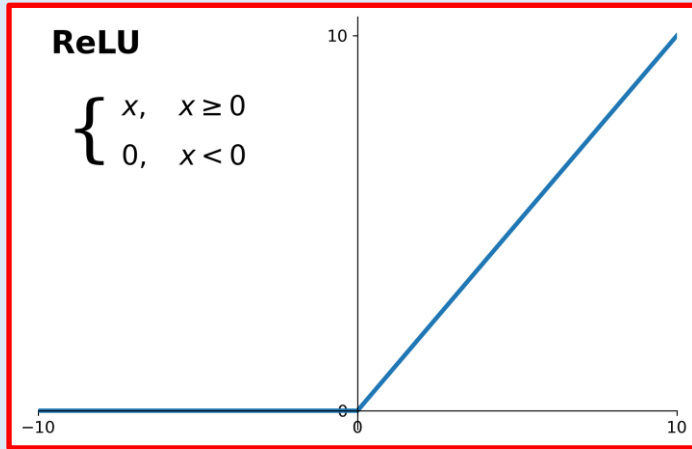
Q: What if we try to build a neural network without one?

$$f = W_2 W_1 x \quad W_3 = W_2 W_1 \in \mathbb{R}^{C \times H}, f = W_3 x$$

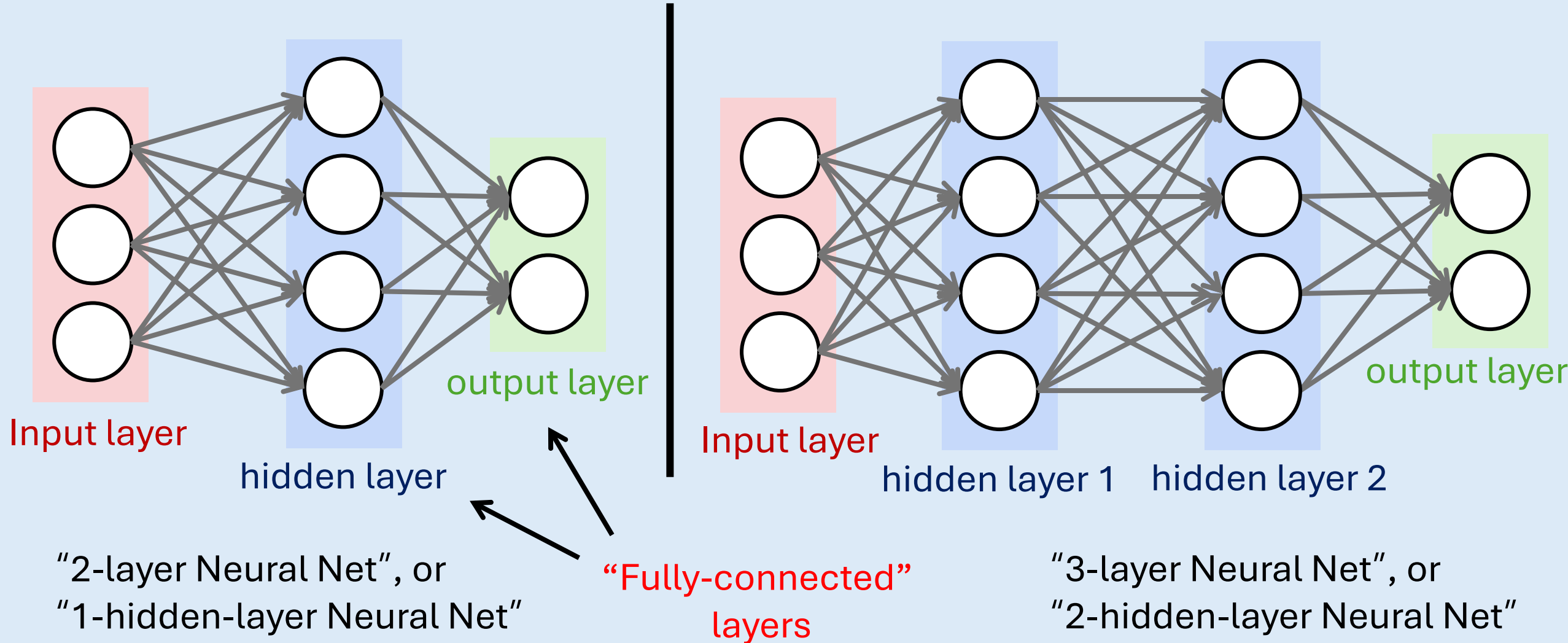
A: We end up with a linear classifier again!

Activation functions

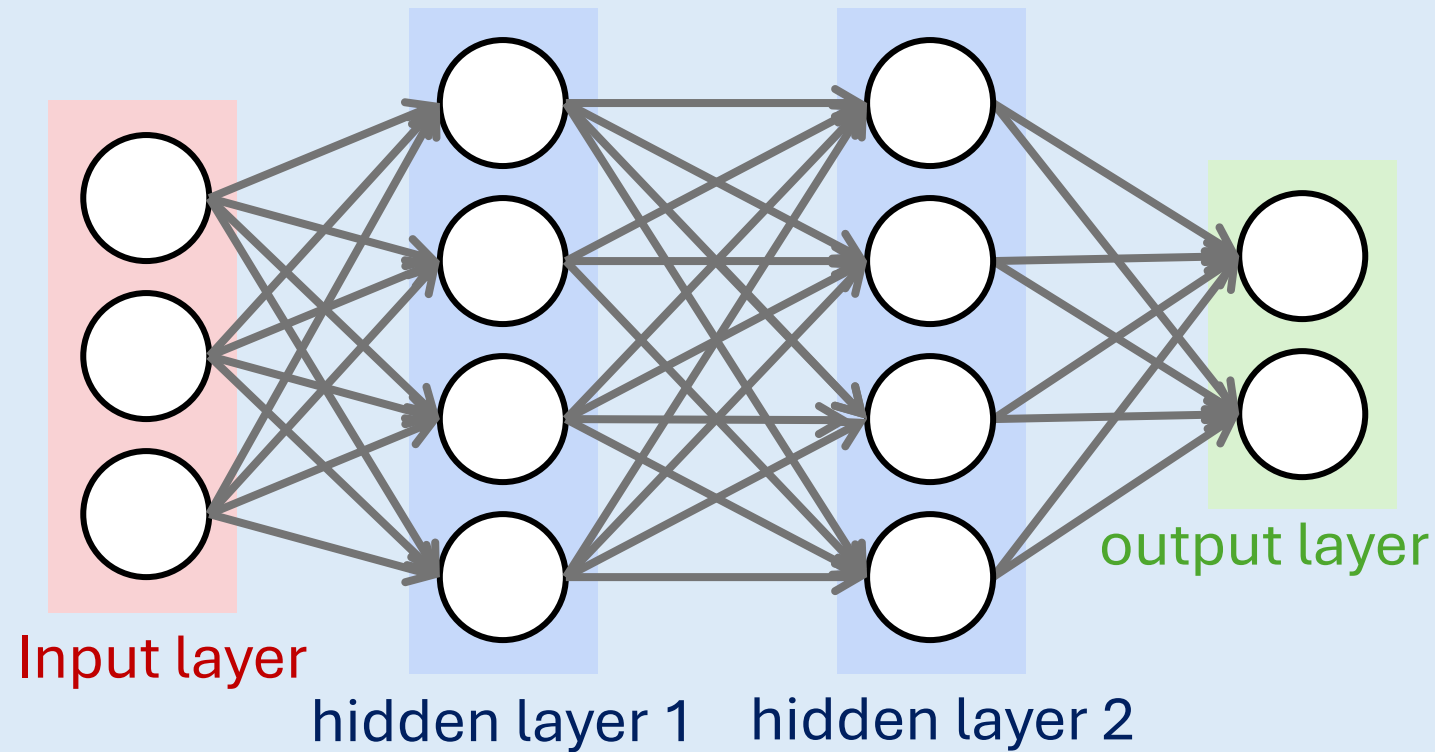
ReLU is a good default choice for most problems



Activation functions



Example feed-forward computation of a neural network



```
# Forward pass of a 3-layer neural network
f = lambda x: 1.0 / (1.0 + np.exp(-x)) # Activation function (sigmoid)
x = np.random.randn(3, 1) # Random input vector (3x1)
h1 = f(np.dot(W1, x) + b1) # First hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # Second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # Output neuron (1x1)
```

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9      h = 1 / (1 + np.exp(-x.dot(w1)))
10     y_pred = h.dot(w2)
11     loss = np.square(y_pred - y).sum()
12     print(t, loss)
13
14     grad_y_pred = 2.0 * (y_pred - y)
15     grad_w2 = h.T.dot(grad_y_pred)
16     grad_h = grad_y_pred.dot(w2.T)
17     grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19     w1 -= 1e-4 * grad_w1
20     w2 -= 1e-4 * grad_w2
```

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9      h = 1 / (1 + np.exp(-x.dot(w1)))
10     y_pred = h.dot(w2)
11     loss = np.square(y_pred - y).sum()
12     print(t, loss)
13
14     grad_y_pred = 2.0 * (y_pred - y)
15     grad_w2 = h.T.dot(grad_y_pred)
16     grad_h = grad_y_pred.dot(w2.T)
17     grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19     w1 -= 1e-4 * grad_w1
20     w2 -= 1e-4 * grad_w2
```

Define the network

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9      h = 1 / (1 + np.exp(-x.dot(w1)))
10     y_pred = h.dot(w2)
11     loss = np.square(y_pred - y).sum()
12     print(t, loss)
13
14     grad_y_pred = 2.0 * (y_pred - y)
15     grad_w2 = h.T.dot(grad_y_pred)
16     grad_h = grad_y_pred.dot(w2.T)
17     grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19     w1 -= 1e-4 * grad_w1
20     w2 -= 1e-4 * grad_w2
```

Define the network

Forward pass

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9      h = 1 / (1 + np.exp(-x.dot(w1)))
10     y_pred = h.dot(w2)
11     loss = np.square(y_pred - y).sum()
12     print(t, loss)
13
14     grad_y_pred = 2.0 * (y_pred - y)
15     grad_w2 = h.T.dot(grad_y_pred)
16     grad_h = grad_y_pred.dot(w2.T)
17     grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19     w1 -= 1e-4 * grad_w1
20     w2 -= 1e-4 * grad_w2
```

Define the network

Forward pass

Calculate the analytical gradients

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9      h = 1 / (1 + np.exp(-x.dot(w1)))
10     y_pred = h.dot(w2)
11     loss = np.square(y_pred - y).sum()
12     print(t, loss)
13
14     grad_y_pred = 2.0 * (y_pred - y)
15     grad_w2 = h.T.dot(grad_y_pred)
16     grad_h = grad_y_pred.dot(w2.T)
17     grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19     w1 -= 1e-4 * grad_w1
20     w2 -= 1e-4 * grad_w2
```

Define the network

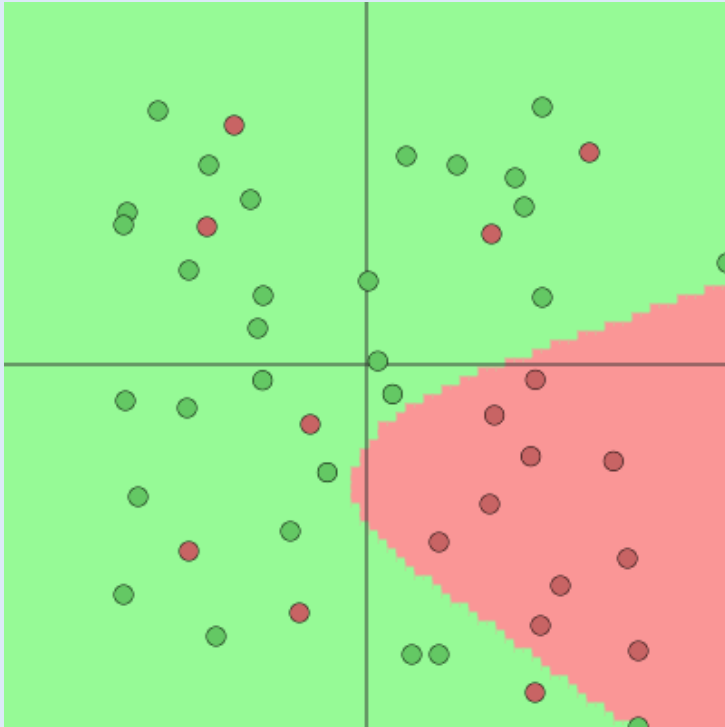
Forward pass

Calculate the analytical gradients

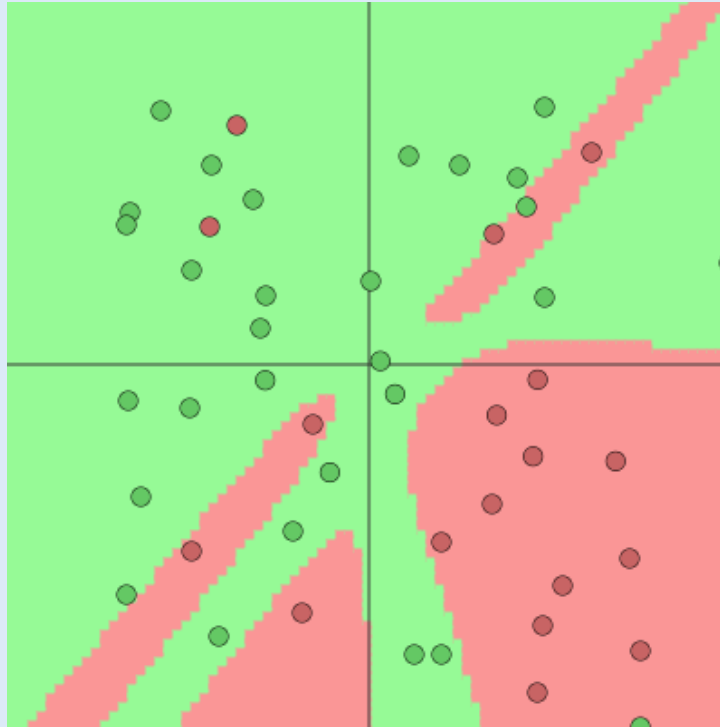
Gradient descent

Setting the number hidden neurons (in 2-layer NN)

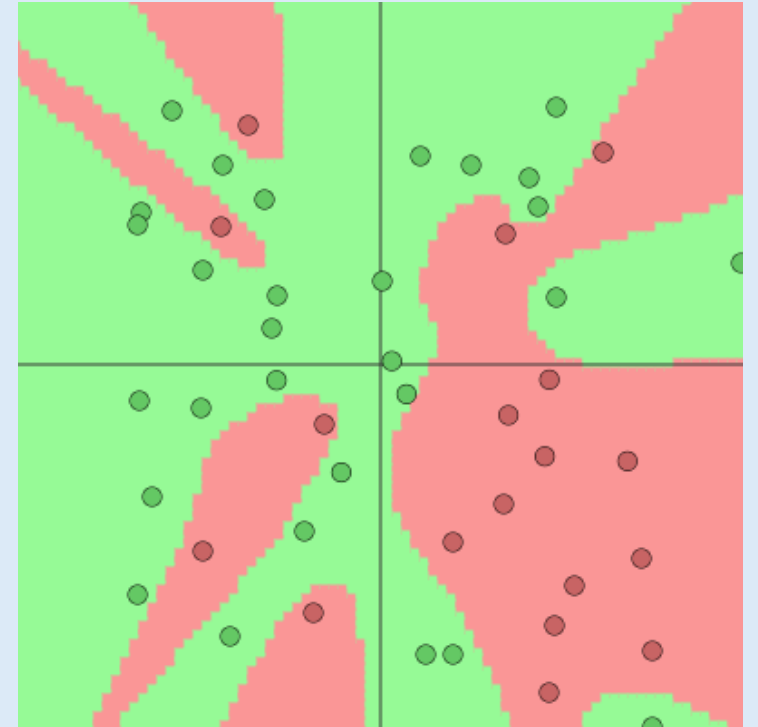
3 hidden neurons



7 hidden neurons



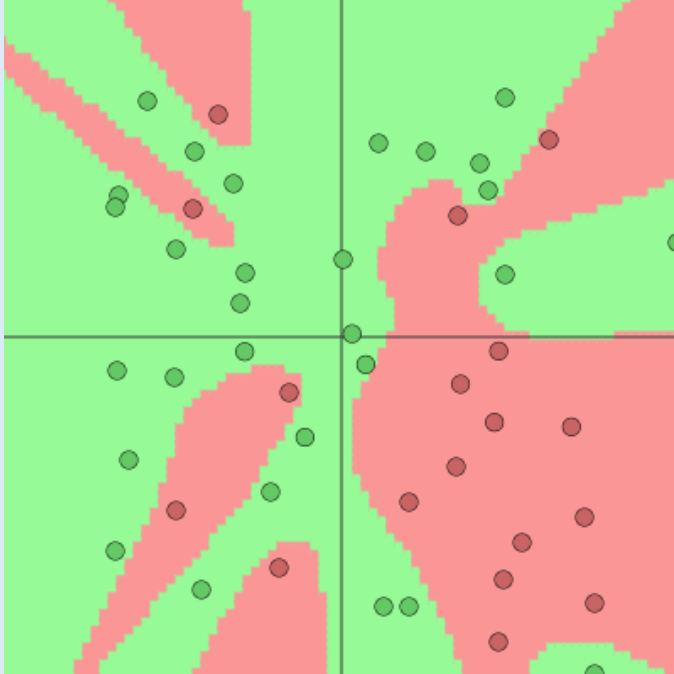
20 hidden neurons



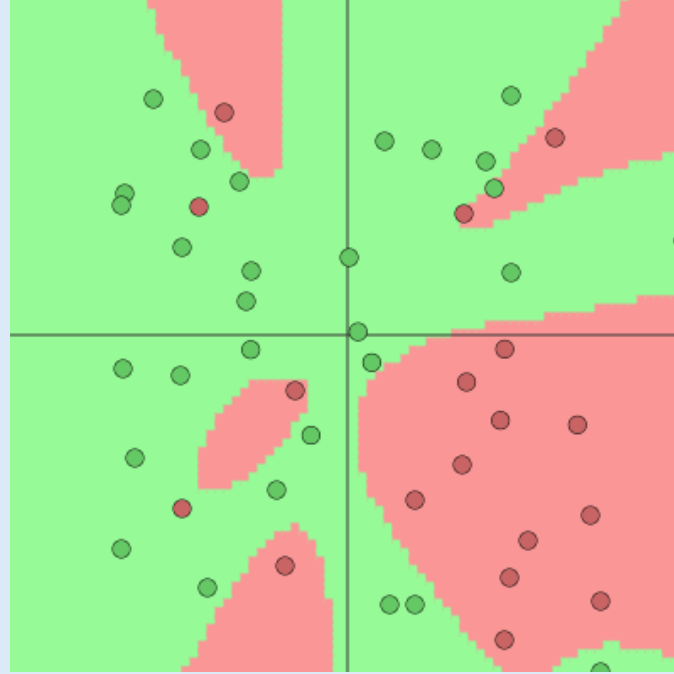
more neurons = more capacity

Full implementation of training a 2-layer Neural Network needs ~20 lines:

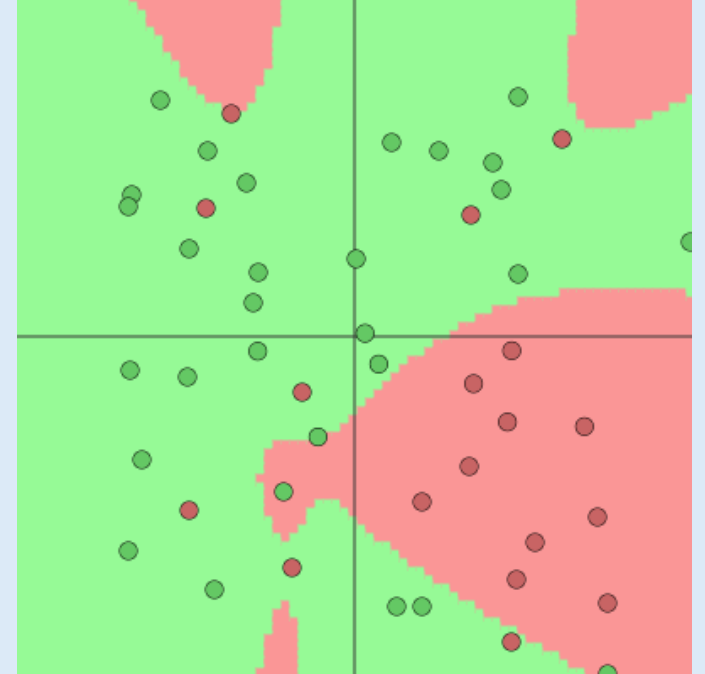
$\lambda = 0.001$



$\lambda = 0.05$



$\lambda = 0.1$



(Web demo with ConvNetJS:

<http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>)

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i) + \lambda R(W)$$

Plugging in neural networks with loss functions

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$

Nonlinear score function

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2)$$

Hinge loss on predictions

$$R(W) = \sum_k W_k^2$$

Regularization

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Total loss: data loss + regularization

Plugging in neural networks with loss functions

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2) \quad \text{Hinge loss on predictions}$$

$$R(W) = \sum_k W_k^2 \quad \text{Regularization}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{Total loss: data loss + regularization}$$

If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ then we can learn W_1 and W_2

(Bad) Idea: Derive $\nabla_W L$ on paper

$$s = f(x; W) = Wx$$

$$\begin{aligned} L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x - W_{y_i,:} \cdot x + 1) \end{aligned}$$

$$\begin{aligned} L &= \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2 \\ &= \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x - W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \end{aligned}$$

$$\nabla_W L = \nabla_W \left(\frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x - W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)$$

Problem: Very tedious: Lots of matrix calculus, need lots of paper

Problem: What if we want to change loss? E.g. use softmax instead of hinge? Need to re-derive everything from scratch!

Problem: Not feasible for very complex models!

References

- Stanford CS231n
- Mathematical Foundations of Deep Neural Networks at Seoul National University.

